

Arquitetura de Software

Projeto: EscolaAlerta

Dupla: Mateus Farias Bentes e Mariana Idalina Bezerra Vidinho

1. Arquitetura da Solução e Padrões de Projeto

1.1 Arquitetura da Solução

O EscolaAlerta segue uma arquitetura em camadas (layered architecture), com separação clara entre apresentação, aplicação e dados. Essa escolha favorece a manutenibilidade e a testabilidade do sistema, características importantes para um projeto desenvolvido por uma dupla, em que cada camada pode ser implementada e testada de forma isolada. O aplicativo Flutter consome a API por meio de requisições REST em JSON; a API concentra toda a lógica de negócio, incluindo o motor de correlação temporal que gera os alertas; e o PostgreSQL persiste os dados de alunos, indicadores e histórico de intervenções.

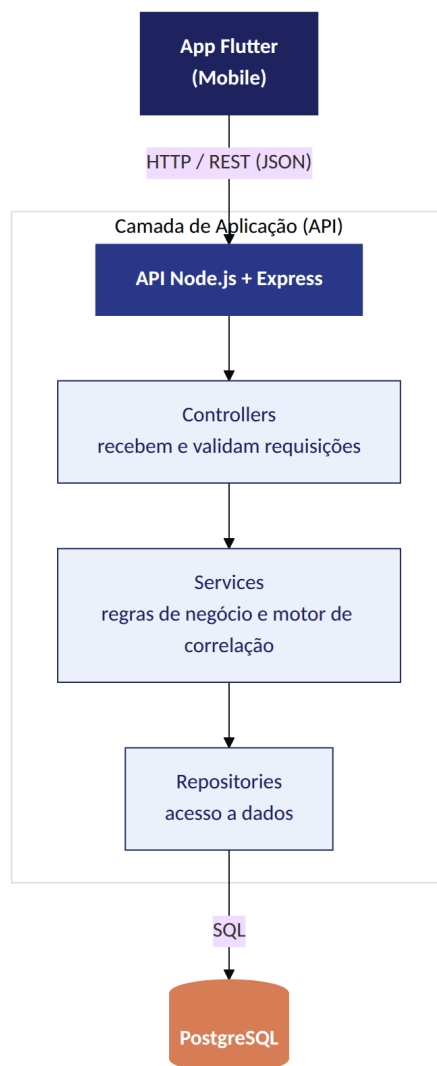


Figura 1: Arquitetura em camadas do EscolaAlerta

Camada	Responsabilidade	Tecnologia
App Flutter (Mobile)	Interface com o usuário (pai/mãe, escola). Consome a API via REST.	Flutter/Dart
Controllers	Recebem as requisições HTTP, validam entrada e delegam para os Services.	Node.js + Express
Services	Regras de negócio: motor de correlação temporal e módulo de nudge.	Node.js
Repositories	Isolam o acesso a dados; convertem objetos de domínio em consultas SQL.	Node.js (pg)
Banco de Dados	Armazena alunos, indicadores, alertas e histórico de intervenções.	PostgreSQL

1.2 Padrões de Projeto (Design Patterns)

Um design pattern é uma solução reutilizável para um problema recorrente de design de software, documentada de forma independente de linguagem de programação (GAMMA et al., 1994). A seguir são justificados os três padrões escolhidos para o EscolaAlerta, cada um associado a um problema concreto identificado nos requisitos do sistema.

1.2.1 Strategy: Motor de Regras (Correlação Temporal)

Problema: o motor de correlação precisa avaliar diferentes tipos de indicador (notas, faltas, ocorrências) e permitir que novos critérios de risco sejam adicionados no futuro, sem exigir alterações no código já existente.

Solução aplicada: cada critério de avaliação é implementado como uma estratégia independente (RegraNotas, RegraFaltas, RegraOcorrencias), todas seguindo a mesma interface. O MotorDeRegras atua como contexto, combinando o resultado das estratégias ao longo do tempo para decidir se um alerta deve ser disparado.

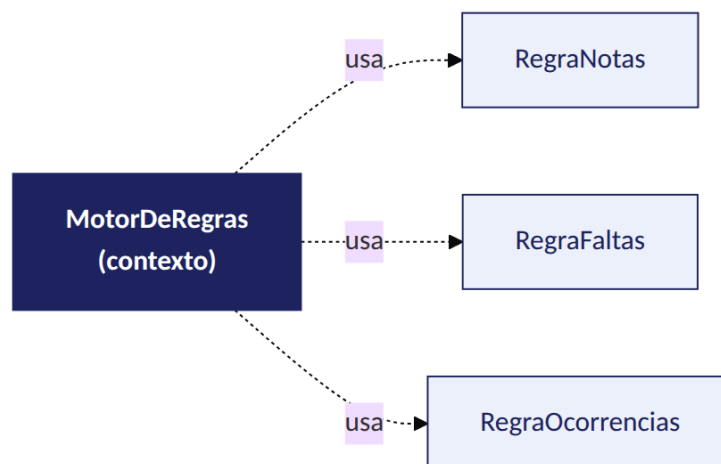


Figura 2: Strategy Pattern aplicado ao motor de regras.

Por que este padrão?: a alternativa mais simples seria uma sequência de condicionais (if/else) dentro do motor de regras. Isso funcionaria para o escopo atual, mas cada novo indicador aumentaria a complexidade de uma única função, dificultando testes e manutenção. Com Strategy, cada regra é testada isoladamente e novos critérios são incluídos sem tocar no motor existente, o

que é especialmente relevante porque o motor de correlação é o principal diferencial técnico do projeto.

1. 2.2 Repository: Camada de Acesso a Dados

Problema: as regras de negócio não devem depender diretamente de comandos SQL, sob risco de acoplar toda a lógica do sistema ao PostgreSQL e dificultar testes automatizados.

Solução aplicada: toda operação de leitura ou escrita no banco passa por um Repository (ex.: AlunoRepository), responsável exclusivamente por converter objetos de domínio em consultas SQL e vice-versa. Os Services conhecem apenas a interface do repositório, nunca o banco em si.



Figura 3: Repository Pattern aplicado à camada de dados.

Por que este padrão?: é uma prática consolidada em arquiteturas de API REST (FOWLER, 2002) e de baixo custo de implementação. Isolar o acesso a dados permite trocar detalhes de armazenamento sem afetar as regras de negócio, e possibilita testar os Services com dados simulados (mocks), sem precisar de um banco real.

1. 2.3 Observer: Módulo de Intervenção Orientada

Problema: quando o motor de regras dispara um alerta, mais de uma ação pode precisar acontecer (notificar a família, atualizar o painel da escola e, futuramente, enviar e-mail), sem que o motor de regras precise conhecer cada um desses destinos.

Solução aplicada: o disparo de um alerta funciona como um sujeito observável, e cada canal de resposta (notificação push, atualização de painel, envio de e-mail) é um observador independente, registrado para reagir ao evento sem se comunicar diretamente com o motor de regras.

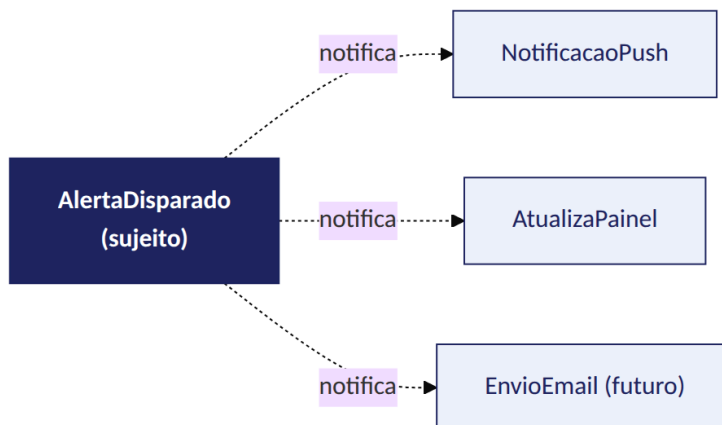


Figura 4: Observer Pattern aplicado ao módulo de nudge.

Por que este padrão?: sem esse desacoplamento, cada novo canal de notificação exigiria alterar o motor de regras diretamente. Com Observer, um novo canal é adicionado apenas registrando mais um observador, o que é importante porque o módulo de intervenção orientada é o diferencial inovador do EscolaAlerta e deve poder crescer (novos tipos de sugestão, novos canais) sem reescrever o motor de correlação.

1.3 Considerações Finais

A combinação de uma arquitetura em camadas com os três padrões descritos organiza o sistema em torno das suas partes mais críticas: o motor de correlação (Strategy), o acesso a

dados (Repository) e a entrega das intervenções orientadas (Observer). Essa estrutura será revisada e detalhada durante a modelagem do sistema, podendo incorporar outros padrões à medida que novos requisitos forem especificados.

2. Plano e Execução de Testes de Software

2.1 Estratégia de Testes

A estratégia segue a pirâmide de testes: a maior parte dos casos é unitária (rápida e barata de manter), um conjunto menor cobre a integração entre camadas da API, e poucos casos de sistema validam o fluxo completo: app Flutter, API e banco de dados funcionando juntos. Essa proporção reduz o tempo de execução da suíte sem abrir mão da cobertura sobre o comportamento observável pelo usuário.



Figura 5: Pirâmide de testes aplicada ao EscolaAlerta.

Cada caso de teste abaixo já nasce com status "Pendente"; atualize para "Aprovado" ou "Reprovado" (com uma nota sobre a falha) conforme os testes forem executados. Assim o mesmo documento serve como plano e como registro da execução.

2.1.1 Testes Unitários

Escopo: classes e funções isoladas, sem acesso a banco de dados ou rede, principalmente as estratégias do motor de regras (Strategy) e funções de validação.

Ferramentas: Jest para a API (Node.js); flutter_test para os widgets e funções do app Flutter.

ID	Unidade testada	Cenário / resultado esperado	Requisito	Status
UT01	RegraNotas.avaliar()	Aluno com média abaixo do limite em 2 disciplinas consecutivas deve ser sinalizado como risco.	RF07	Pendente
UT02	RegraFaltas.avaliar()	Aluno com faltas acima do limite no período deve ser sinalizado como risco.	RF05 / RF07	Pendente
UT03	RegraOcorrencias.avaliar()	Aluno com 3 ou mais ocorrências no mês deve ser sinalizado como risco.	RF04 / RF07	Pendente
UT04	MotorDeRegras.classificar()	Combinação de múltiplos sinais de risco deve retornar o nível correto (baixo, moderado ou alto).	RF07	Pendente
UT05	MotorDeRegras.classificar()	Nenhum sinal de risco presente deve retornar nível "baixo".	RF07	Pendente
UT06	Validação do cadastro de aluno	Requisição sem CPF ou dado obrigatório deve retornar erro de	RF02	Pendente

ID	Unidade testada	Cenário / resultado esperado	Requisito	Status
		validação (400), sem chamar o Repository.		
UT07	Hash de senha (bcrypt)	A senha em texto puro nunca deve ser persistida; apenas o hash deve ser gerado e comparado.	RNF01	Pendente
UT08	GeradorDePlanoDeAcao	A partir de uma causa raiz de risco (ex.: faltas), deve retornar a sugestão pedagógica correspondente.	RF12	Pendente

2.1.2 Testes de Integração

Escopo: comunicação entre Controllers, Services e Repositories, e destes com um banco de dados de teste (PostgreSQL isolado ou em contêiner), sem envolver o app Flutter.

Ferramentas: Jest + Supertest, executando requisições HTTP reais contra a API em ambiente de teste.

ID	Endpoint / fluxo testado	Cenário / resultado esperado	Requisito	Status
IT01	POST /alunos	Cria um aluno vinculado a um responsável autenticado e persiste corretamente no banco de teste.	RF02	Pendente
IT02	POST /notas + GET /alunos/:id/notas	Nota registrada é recuperada com os mesmos valores de disciplina, período e nota.	RF03	Pendente
IT03	POST /login (credenciais válidas)	Retorna token JWT válido para um responsável cadastrado.	RF01 / RNF01	Pendente
IT04	POST /login (credenciais inválidas)	Retorna erro 401 e nenhum token é emitido.	RF01 / RNF01	Pendente
IT05	Registro de faltas recorrentes	Ao ultrapassar o limite configurado, o motor de regras dispara um alerta persistido no banco.	RF05 / RF07 / RF08	Pendente
IT06	GET /alertas/:alunoid	Retorna o histórico de evolução de risco armazenado, em ordem cronológica.	RF08	Pendente
IT07	POST /alertas/:id/feedback	Registra o feedback de resolução do responsável e realimenta o histórico do aluno.	RF13	Pendente

2.1.3 Testes de Sistema

Escopo: fluxos completos do ponto de vista do usuário, com o app Flutter, a API e o banco de dados operando em conjunto.

Abordagem: execução roteirizada (manual) a partir dos cenários abaixo, com possibilidade de automatizar os principais fluxos futuramente com o pacote `integration_test` do Flutter.

ID	Cenário	Resultado esperado	Requisito	Status
ST01	Login, cadastro do filho e painel inicial	Responsável faz login, cadastra o aluno e visualiza o painel sem alertas pendentes.	RF01 / RF02 / RF08	Pendente
ST02	Alerta automático por faltas	Escola registra faltas consecutivas; responsável recebe alerta no app com o nível de risco correspondente.	RF05 / RF07 / RF08 / RNF06	Pendente

ID	Cenário	Resultado esperado	Requisito	Status
ST03	Plano de ação e feedback	Alerta gerado exibe o plano de ação sugerido; responsável registra feedback de resolução com sucesso.	RF12 / RF13 / RNF07	Pendente
ST04	Controle de acesso entre contas	Responsável autenticado não consegue visualizar dados de aluno vinculado a outra conta.	RNF01 / RNF02	Pendente
ST05	Uso em mobile e web	As principais telas do EscolaAlerta funcionam corretamente tanto na versão mobile quanto na web (mesma base Flutter).	RNF05	Pendente

2.2 Rastreabilidade Requisito × Teste

A tabela a seguir relaciona cada requisito do EscolaAlerta aos casos de teste que o cobrem, permitindo verificar a cobertura antes de cada entrega. Requisitos ainda sem teste associado devem ser detalhados na próxima rodada de planejamento.

Requisito	Descrição	Casos de teste
RF01	Autenticar responsáveis via login seguro	IT03, IT04, ST01
RF02	Cadastrar alunos vinculados aos responsáveis	UT06, IT01, ST01
RF03	Registrar notas por disciplina e período	IT02
RF04	Registrar chamadas de atenção (ocorrências)	UT03
RF05	Registrar frequência e faltas	UT02, IT05, ST02
RF06	Registrar saídas antecipadas	(a detalhar na próxima rodada de testes)
RF07	Aplicar motor de regras e classificar o risco em baixo, moderado ou alto	UT01, UT02, UT03, UT04, UT05, IT05, ST02
RF08	Gerar alertas automáticos e exibir painel com histórico de evolução	IT05, IT06, ST01, ST02
RF12	Gerar plano de ação a partir da causa raiz do alerta	UT08, ST03
RF13	Registrar feedback de resolução e realimentar o histórico	IT07, ST03
RNF01	Segurança: autenticação JWT e senha criptografada (bcrypt)	UT07, IT03, IT04, ST04
RNF02	Conformidade com a LGPD: dados fictícios e controle de acesso	ST04
RNF03	Manutenibilidade: código modular, versionado em Git/GitHub	Verificação de processo (revisão de código), não coberta pelos testes automatizados
RNF04	Precisão: motor de regras avaliado frente a cenários simulados (meta: 80% de acerto)	UT01–UT05 (conjunto de cenários simulados)
RNF05	Disponibilidade: aplicação web e mobile, via Flutter	ST05
RNF06	Desempenho: API REST com bom tempo de resposta	ST02
RNF07	Explicabilidade: alerta e sugestão rastreáveis à regra que os originou	ST03

2.3 Plano de Execução

Os testes unitários são executados a cada alteração de código, antes de qualquer integração ao repositório principal, idealmente de forma automatizada a cada commit. Os testes de

integração são executados ao final de cada sprint, antes de mover os itens correspondentes para "Testado" no quadro Kanban do projeto. Os testes de sistema são executados antes de cada entrega para a professora e, de forma completa, antes da apresentação final à banca.

2.4 Considerações Finais

Este plano cobre os requisitos funcionais e não funcionais já levantados no pré-projeto, com ênfase no motor de correlação e no módulo de nudge, que concentram o maior risco técnico do EscolaAlerta. A cada nova funcionalidade implementada, novos casos devem ser adicionados nos três níveis, mantendo a tabela de rastreabilidade atualizada.

3. Diagramas UML

Este capítulo reúne os diagramas UML que detalham, do ponto de vista estrutural e comportamental, o funcionamento do EscolaAlerta: o diagrama de caso de uso, que mapeia atores e funcionalidades; o diagrama de classes, que detalha as entidades do domínio e seus relacionamentos; e o diagrama de sequência, que descreve o fluxo de geração de um alerta desde o registro de um indicador até a resposta do responsável.

3.1 Diagrama de Caso de Uso

O diagrama de caso de uso delimita o Sistema EscolaAlerta e mapeia como cada ator interage com suas funcionalidades. Dois atores primários foram modelados: Orientador escolar e Responsável, ambos generalizando o ator abstrato Usuário do sistema, que concentra o caso de uso Autenticar-se, comum a qualquer perfil de acesso.

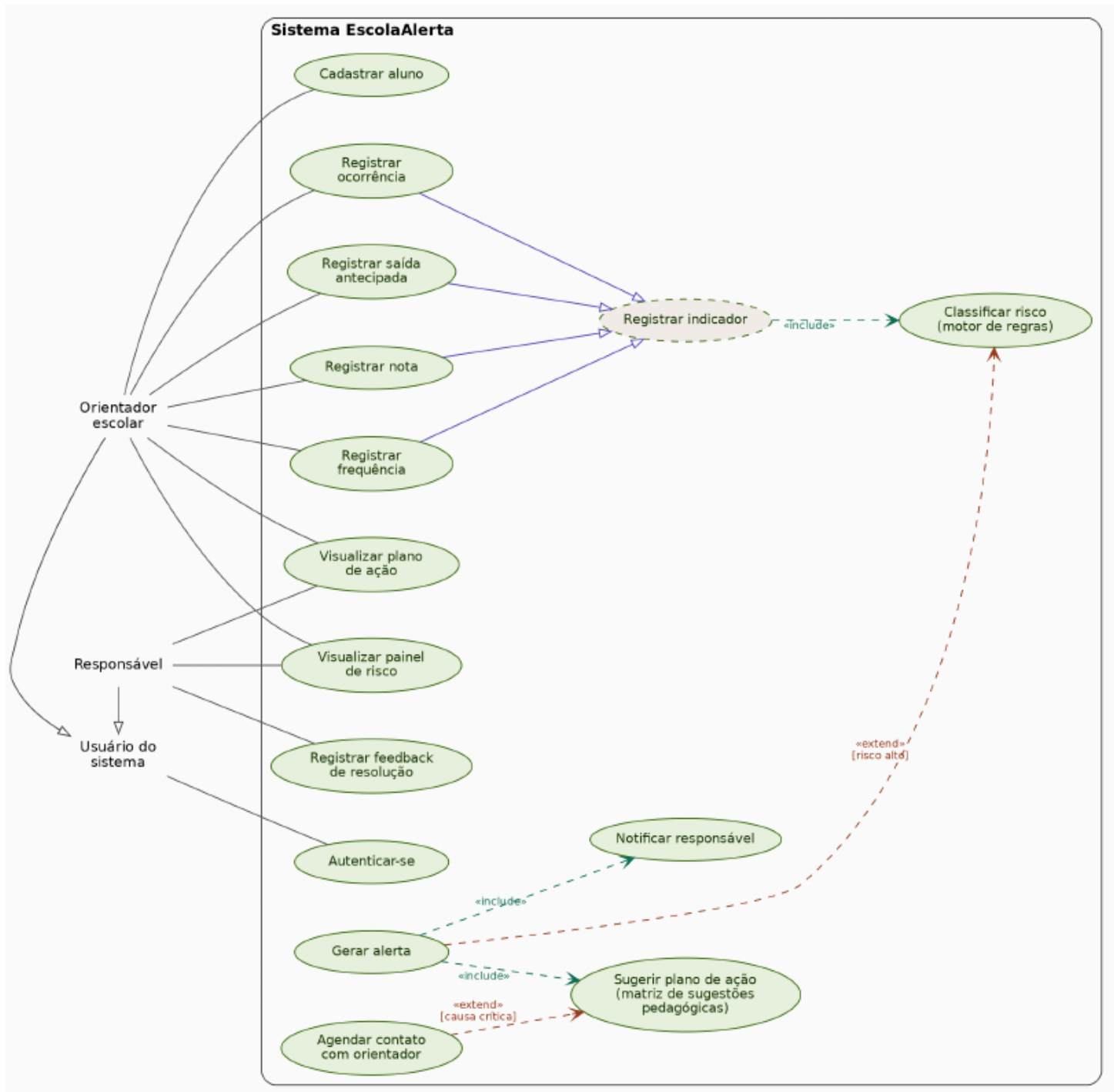


Figura 6: Diagrama de caso de uso do EscolaAlerta.

Descrição da Figura 6: O Orientador escolar é responsável por cadastrar alunos e registrar os indicadores de acompanhamento (ocorrência, saída antecipada, nota e frequência); todos esses registros incluem («include») o caso de uso Registrar indicador, que por sua vez inclui Classificar risco (motor de regras). O Responsável, por sua vez, apenas visualiza o plano de ação e o painel de risco e registra feedback de resolução, ele não registra indicadores diretamente, refletindo a decisão de projeto de que só a escola alimenta o motor de correlação. O caso de uso Gerar alerta inclui Notificar responsável e Sugerir plano de ação, e é estendido («extend») por condições específicas: risco alto dispara a notificação automaticamente, e uma causa crítica identificada estende a sugestão do plano de ação. Essa separação de responsabilidades entre os dois atores é o que garante a rastreabilidade e a governança dos dados que alimentam o motor de regras.

3.2 Diagrama de Classes

O diagrama de classes detalha as entidades do domínio do EscolaAlerta e os relacionamentos entre elas, servindo de base para o modelo de dados da API.

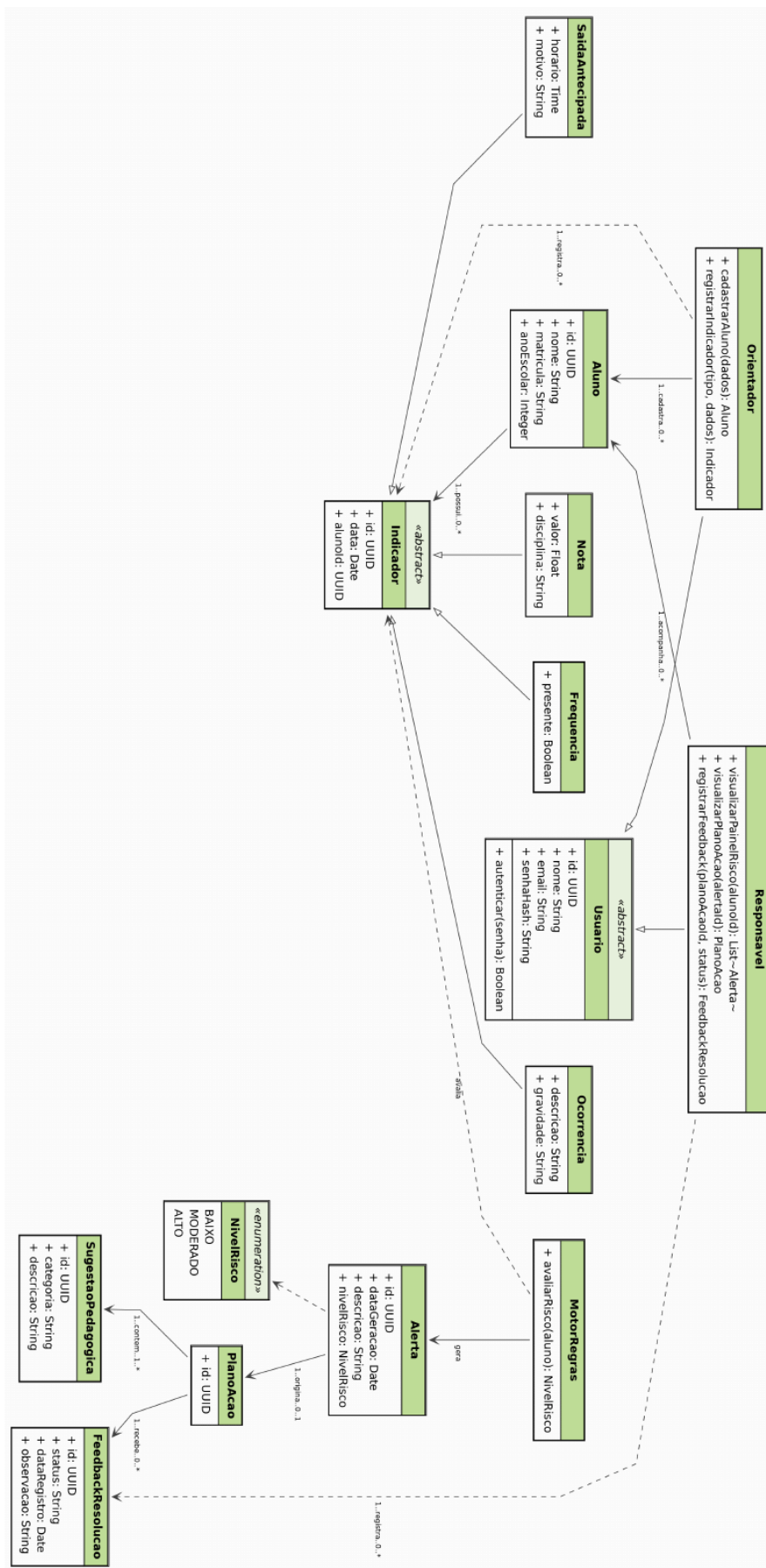


Figura 7: Diagrama de classes do EscolaAlerta.

Descrição da Figura 7: As classes Orientador e Responsavel concentram os métodos de negócio de cada ator (cadastrarAluno, registrarIndicador, visualizarPainelRisco, visualizarPlanoAcao, registrarFeedback) e generalizam a classe abstrata Usuario, que guarda os dados comuns de autenticação (id, nome, email, senhaHash). A classe Aluno é associada a um Orientador (que a cadastra) e a um Responsavel (que a acompanha), e possui vários registros da classe abstrata Indicador, especializada em Nota, Frequencia, Ocorrencia e SaidaAntecipada. A classe MotorRegras avalia o risco do aluno e gera um Alerta, que carrega um NivelRisco (enumeração BAIXO/MODERADO/ALTO) e pode originar um PlanoAcao, este contém uma ou mais SugestaoPedagogica e recebe FeedbackResolucao registrados pelo responsável. A estrutura reflete diretamente os padrões de projeto descritos na Parte 1: Indicador como hierarquia avaliada por estratégias, e Alerta como o evento que dispara as notificações do módulo Observer.

3.3 Diagrama de Sequência

O diagrama de sequência descreve o fluxo completo de geração de um alerta, do registro do indicador pela escola até o feedback do responsável, incluindo o ramo alternativo em que o risco calculado é baixo.

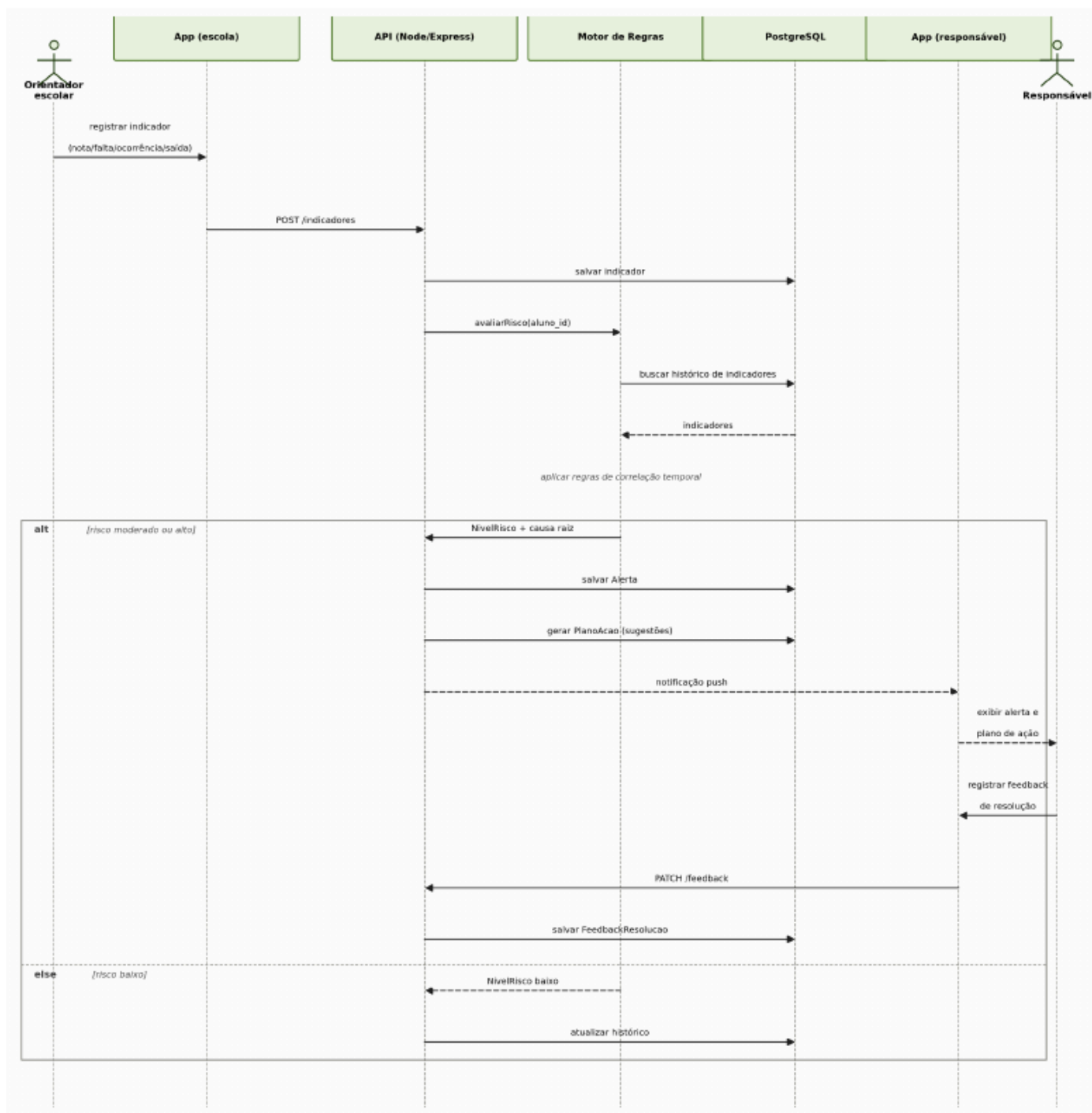


Figura 8: Diagrama de sequência: registro de indicador e geração de alerta.

Descrição da Figura 8: O fluxo se inicia quando o Orientador escolar registra um indicador (nota, falta, ocorrência ou saída) pelo App (escola), que envia um POST /indicadores para a API. A API salva o indicador no PostgreSQL e aciona avaliarRisco(aluno_id) no Motor de Regras, que busca o histórico de indicadores no banco e aplica as regras de correlação temporal. No ramo alt [risco moderado ou alto], o motor retorna o nível de risco e a causa raiz; a API salva o Alerta, gera o PlanoAcao com sugestões e notifica o App (responsável) por push, que exibe o alerta e o plano de ação e permite ao responsável registrar feedback de resolução, persistido via PATCH /feedback. No ramo else [risco baixo], apenas o histórico do aluno é atualizado, sem notificação. Esse diagrama evidencia como os três padrões de projeto (Strategy, Repository e Observer) se articulam em tempo de execução.